

WORLD IN POCKET · NOTA TECNICA INTERNA

Infrastruttura geografica dell'audioguida

Come si passa da "un punto e un raggio generico" a "un ingresso vero, un raggio calibrato sull'edificio reale, e una posizione utente corretta sulla strada" — le cinque fasi della pipeline, i numeri di oggi, e i passaggi per riprodurla da zero.

3.152.393	55.602	1.431.998	94,5 GB
EDIFICI MONDO ESTRATTI	POI IT CON RAGGIO CALIBRATO	BENI NELL'ATLANTE	PHOTON IN SCARICAMENTO

§0 Il problema che si sta risolvendo

Perché tutta questa infrastruttura, in due frasi
Fino a poco tempo fa ogni POI aveva un solo dato di posizione — il **centroide** — e un raggio di trigger **fisso per categoria** (es. +40m per "castello", uguale per una torretta o una fortezza da 300 metri di lato). Per edifici grandi o irregolari il centroide geometrico cade spesso lontano da dove cammina davvero la gente: l'utente arriva al portone ma resta fuori dal cerchio, e l'audioguida non parte mai.

L'obiettivo di questa infrastruttura è sostituire quel punto/raggio arbitrario con dati reali per ogni POI: **dov'è davvero il perimetro dell'edificio, dov'è il vero ingresso, e dove si trova davvero l'utente rispetto alla strada** — così il trigger scatta esattamente quando si arriva, non prima, non dopo, non mai.

1

Il perimetro reale di ogni edificio, non un centroide

Ogni POI viene abbinato al poligono del suo edificio reale, da cui si calcola un **raggio equivalente** — la distanza massima dal centro a un vertice del perimetro (non il perimetro/area grezzi: quelli sono in gradi, non metri, serve la conversione haversine).

```
-- raggio dal perimetro OSM (max distanza centro→vertice)
CAST(max(111320.0 * sqrt(
    power(p.lat - c.clat, 2) +
    power((p.lon - c.clon) * cos(radians(c.clat))), 2)
)) AS INT) AS radius_m

-- raggio calibrato finale (clamp prudenziale)
geofence_radius = clamp(radius_m + 15, 25, 250)
alert_radius     = clamp((radius_m + 15) * 2.5, 120, 500)
```

Due fonti, in cascata

25.373

OpenStreetMap · wikidata
esatto (57) +
spaziale+nome fuzzy ≥86%
(25.316)

30.229

Microsoft ML · fallback
SOLO spaziale, solo
categorie-edificio, sui
rimasti senza match OSM

55.602 /

~106.000
copertura reale POI-edificio
Italia (52,4%, era 24% una
settimana fa)

Microsoft Global ML Building Footprints (edifici rilevati via satellite, 15,6M poligoni caricati per l'Italia) copre molto di più di OSM ma non ha nomi — il match è *solo* spaziale, quindi va ristretto alle categorie che sono davvero edifici (chiesa, museo, castello, monumento...). Provato senza quel filtro: parchi e giardini venivano abbinati a capanni vicini a caso — geometricamente plausibile, concettualmente senza senso.

- **Estrazione mondo** — `scratch/footprint-pilot/extract-world-nodes-parts.mjs` · 15 parti del planet.pbf, checkpoint per parte + sotto-parte, sopravvive ai crash
- **Match Italia (OSM)** — `match-pois.mjs` + `apply-italy.mjs` · sola lettura poi scrittura con rollback
- **Match Italia (Microsoft)** — `ms-buildings-italy.mjs` · 158 tile per-paese, nessuno scan globale

LEZIONE INGEGNERISTICA

Un dataset da 95GB (il planet) non entra in RAM: ogni rase è spezzata in parti/blocchi checkpointati singolarmente (`memory_limit` 1-2GB, sotto-divisione automatica delle parti dense), così un crash costa al massimo un blocco, mai ore di lavoro.

2

Ingresso

ITALIA APPLICATO

GERARCHIA COMPLETA DA SCRIVERE

Il punto esatto dove ancorare il trigger, non il centro dell'edificio
Quando esiste, il trigger si centra sul nodo OSM taggato come varco d'accesso
— ma solo se il tag indica davvero un ingresso pubblico:

TAG OSM	USO	MOTIVO
main	✓ priorità 1	ingresso principale dichiarato
yes	✓ priorità 2	ingresso generico
secondary / staircase	✓ priorità 3-4	varco pubblico secondario, ancora valido
service / exit / emergency / no	X escluso	porta staff, sola uscita, uscita di sicurezza, o letteralmente "non è un ingresso"

La whitelist esplicita nasce da un bug reale trovato in verifica manuale: il filtro permissivo iniziale abbinava un museo al tag `exit` di un'uscita e una terrazza panoramica a un varco taggato `no`.

Copertura Italia oggi: **2.077 POI** con ingresso da tag OSM (2.067 main/yes + 10 secondary/staircase) — una piccola minoranza rispetto ai 55.602 con perimetro, perché la maggior parte degli edifici OSM ha il contorno disegnato ma non il portone taggato.

Quando il tag non c'è — la gerarchia prevista

- 1. tag `entrance` OSM (sopra)
- 2. nodo civico OSM più vicino

- 3. **indirizzo** → **strada**: la via dell'indirizzo cercata tra le strade con nome delle road tiles entro ~150m — l'ingresso diventa il punto del perimetro rivolto verso quella via (mai sul retro)
- 4. solo per le chiese: lato piazza/pedonale, altrimenti lato ovest
- 5. strada vera più vicina (le road tiles escludono di proposito le vie di servizio)

Script `match-entrances-italy.mjs`: non ancora scritto, dipende dalle road tiles Italia (fase 3) e dagli indirizzi (fase 5).

3

Road tiles

ITALIA IN CARICAMENTO

MONDO IN CODA

Il grafo stradale: corregge il GPS e orienta l'ingresso, non calcola percorsi (ancora)
Due usi distinti, nessuno dei due è "un navigatore":

Snap-to-path

la posizione GPS grezza (rumorosa tra edifici alti) viene agganciata alla strada reale più vicina — corregge il *rumore*, non calcola un tragitto

Lato-ingresso

i nomi via del grafo permettono di orientare l'ingresso stimato verso la strada giusta (fase 2)

Due scope, due script

SCOPE	SCRIPT	FASI CHECKPOINTATE	PERCHÉ DIVERSO
Italia	<code>extract-road-tiles-italy.mjs</code>	A · way → B · nodi → C · join a 32 blocchi (way-id % 32)	estratto autosufficiente (2,07GB): way+nodi nello stesso file, un'unica scansione basta
Mondo	<code>extract-road-tiles-world-poicells.mjs</code>	0 · celle POI → W · way/parte → N · nodi/parte → G · join a 8 blocchi	il planet è ordinato nodi→way: le "parti" non sono mai autosufficienti da sole. Inoltre limitato alle sole celle 0,05° che contengono un POI (dilatate +1), non l'intero pianeta

Il design mondo v1 (per-parte, senza le fasi separate) produceva parquet
"completati" ma **vuoti** — 6 parti, 157 byte ciascuna, zero righe — perché il join

way→nodi dentro una singola parte del planet non può mai funzionare strutturalmente. Disattivato, riscritto a due fasi.

27.822

tile Italia (griglia 0,05° × modalità auto/piedi)

321.768

celle-POI mondo dilatate, da 1.879.735 POI

Output: `.json.gz` per tile, caricati sul bucket Storage `road_tiles`, serviti da `GET /api/roads/tile` — stesso formato ovunque, un solo parser client.

4

Atlante beni culturali IN CRESCITA, ~20 PAESI

Layer informativo parallelo — non è (ancora) collegato all'audioguida

1.431.998 beni raccolti da registri patrimoniali nazionali — Italia (MiC/ArCo), Francia (Mérimeé), Paesi Bassi, Danimarca, Austria, Polonia, Germania (3 Länder), Spagna, Regno Unito (4 nazioni), Irlanda, Fiandre, Croazia, Slovenia, Svezia, Norvegia, Estonia, Svizzera — quasi tutti via il connettore Wikidata generico, paginato sul **QID dell'item** (non sull'ID del registro: molti registri usano formati alfanumerici tipo `D-2-73-164-65`, il QID è sempre numerico ed è indipendente dal formato).

Vive in `beni_culturali`: solo punto + indirizzo + classificazione — **nessun perimetro, nessun ingresso, nessuna audioguida**. È per scelta di prodotto: la maggior parte dei beni non è turistica/visitabile, e verrà mostrata da una chip mappa  dedicata con una scheda leggera (nome, tipologia, indirizzo, badge "bene protetto"). Solo il sottoinsieme classificato fascia A (o B confermata) verrà *promosso* a POI vero in `shared_pois` — e solo a quel punto entra nella pipeline delle fasi 1–3 sopra.

5

Indirizzi via Photon INDICE IN SCARICAMENTO

Il tassello che manca per riempire l'indirizzo di ogni bene/POI, senza violare licenze

Serve un geocoder (coordinate ↔ indirizzo) per completare i record senza uno dei due dati. Google e il geocoding Mapbox sono esclusi **per licenza**, non per preferenza: vietano di salvare permanentemente i loro risultati — esattamente l'uso che serve qui (1,4M+ righe). [Photon](#) (komoot), costruito su dati OpenStreetMap (ODbL, nessun vincolo di storage), self-hosted: costo per chiamata zero una volta acceso.

94,5 GB

indice Elasticsearch, in scaricamento via BITS
verso `D:\PHOTON`

181,7 GB

spazio libero disponibile prima di iniziare

Non può girare su Vercel (dove sta oggi `server.ts`): serve un processo sempre acceso con disco persistente — pianificato un Droplet DigitalOcean dedicato (8GB RAM/4vCPU/160GB SSD stimato, traffico basso: riempimento indirizzi in batch + qualche lookup interattivo, non un motore di ricerca pubblico). `server.ts` lo chiamerebbe con lo stesso pattern proxy già usato per Foursquare/TripAdvisor/Groq — un endpoint nuovo, cache in `api_cache`, nessun cambiamento lato client.

NON SOSTITUISCE

La ricerca luoghi/autocomplete già in app (Mapbox/Geoapify) resta com'è — quello è un uso live, non uno storage permanente, non ha lo stesso vincolo di licenza. Photon è specificamente per il riempimento bulk e permanente degli indirizzi mancanti.

6

Come arriva al trigger reale

La regola è sempre "espandi, mai restringere" — zero rischio di regressione

```
// src/lib/guideSettings.ts - stessa logica su Android/iOS nativo
if (footprint?.hasEntrance) {
  trigger = max(default_modalità, footprint.geofenceRadius)
  alert   = max(default_modalità, footprint.alertRadius)
  centro  = entrance_lat/lon    // non centroide
```

```
}
// altrimenti: comportamento identico a prima, invariato
```

Il `max()` — mai `min()` — è la garanzia anti-regressione: un POI piccolo (statua, raggio 3m) resta al default di modalità, un edificio grande viene solo allargato. Dove non c'è ancora un footprint valido, zero differenza rispetto a prima.

WEB

`useGeofencing.ts`

ANDROID

`GeofenceManager.kt`

IOS

`BackgroundPoiManager.swift`

7

Limite noto IDEA FUTURA, NON IMPLEMENTATA

Il trigger è sempre distanza in linea d'aria — mai un percorso reale
 Oggi: un POI a 300m in linea d'aria fa scattare l'alert anche se un fiume, un isolato intero o vie parallele rendono il percorso a piedi molto più lungo — le road tiles correggono solo il *rumore* GPS, non calcolano un tragitto. Proposta discussa e annotata: un router pedonale leggero (A* a corto raggio sul grafo già estratto in fase 3, niente traffico/lunghe distanze — problema molto più semplice di un navigatore generico) per due cose:

- distanza di trigger sul percorso reale, non in linea d'aria
- indicazione vocale del percorso ("gira a sinistra tra 50m") — riusa la pipeline TTS background già esistente, zero infrastruttura nuova
- indicazione visiva su **schermata di blocco**: Android via notifica persistente (estensione del tasto ► di arrivo già esistente), iOS via Live Activity (lavoro nuovo)

Solo a piedi — in auto resta il deep-link a Google/Apple Maps: distanze lunghe, serve il traffico live, un errore è un rischio di sicurezza vero. Sproporzionato costruire un motore di navigazione auto da zero.

§ Riprodurre la tecnologia da zero

La sequenza, indipendentemente dal progetto — cosa serve prima di cosa

- 01 **Procurarsi un estratto OSM completo e non troncato**
planet.pbf per il mondo, o un estratto regionale (Geofabrik) per un'area — **verificare sempre la dimensione attesa**: un file troncato passa i controlli superficiali ma dà zero risultati a valle.
 - 02 **Estrarre i perimetri edificio**
way taggate (building/historic/tourism/amenity di interesse) + semi-join sui nodi referenziati. Su dataset grandi: **mai un'unica query monolitica** — spezzare in parti/blocchi checkpointati, con un tetto di memoria conservativo e sotto-divisione automatica per le zone dense.
 - 03 **Abbinare i perimetri ai POI esistenti**
match esatto su un identificatore condiviso (es. wikidata) quando c'è, poi spaziale+nome (griglia + soglia di similarità testuale) per il resto.
 - 04 **Aggiungere una seconda fonte SOLO spaziale come fallback**
un dataset di edifici ML (es. Microsoft/Google Open Buildings) copre dove OSM non ha disegnato nulla — ma senza nomi il match va ristretto alle categorie che sono davvero edifici, altrimenti produce abbinamenti plausibili ma senza senso.
 - 05 **Calcolare un raggio in METRI, non nell'unità nativa della geometria**
funzioni tipo "perimetro"/"area" di un motore spaziale lavorano in gradi se i dati sono lat/lon — serve sempre una conversione haversine esplicita, altrimenti il raggio risulta 0 silenziosamente.
 - 06 **Estrarre il grafo stradale con nomi via**
stesso principio del punto 2: checkpoint per fase, scope ridotto dove possibile (es. solo celle con POI, non l'intero pianeta) per tenere il problema trattabile.
 - 07 **Usare il grafo per due cose separate**
-

agganciare la posizione utente rumorosa alla strada vera (snap), e stimare il lato-ingresso di un edificio dalla via del suo indirizzo.

- 08 **Riempire gli indirizzi mancanti con un geocoder che permetta lo storage permanente**